

melodic

noetic

Show EOL distros: ☐

Documentation Status

rosterial: [rosterial_client \(/rosterial_client?distro=noetic\)](#) | [rosterial_msgs \(/rosterial_msgs?distro=noetic\)](#) | [rosterial_python \(/rosterial_python?distro=noetic\)](#)

Package Links

- [Tutorials \(/rosterial/Tutorials\)](#)
- [FAQ \(http://answers.ros.org/questions/scope:all/sort:activity-desc/tags:rosterial/page:1/\)](http://answers.ros.org/questions/scope:all/sort:activity-desc/tags:rosterial/page:1/)
- [Changelog \(http://docs.ros.org/en/noetic/changelogs/rosterial/changelog.html\)](http://docs.ros.org/en/noetic/changelogs/rosterial/changelog.html)
- [Change List \(/rosterial/ChangeList\)](#)
- [Reviews \(/rosterial/Reviews\)](#)

Dependencies (4)

Jenkins jobs (10)

Package Summary

✓ Released ✓ Continuous Integration ✓ Documented

Metapackage for core of rosterial.

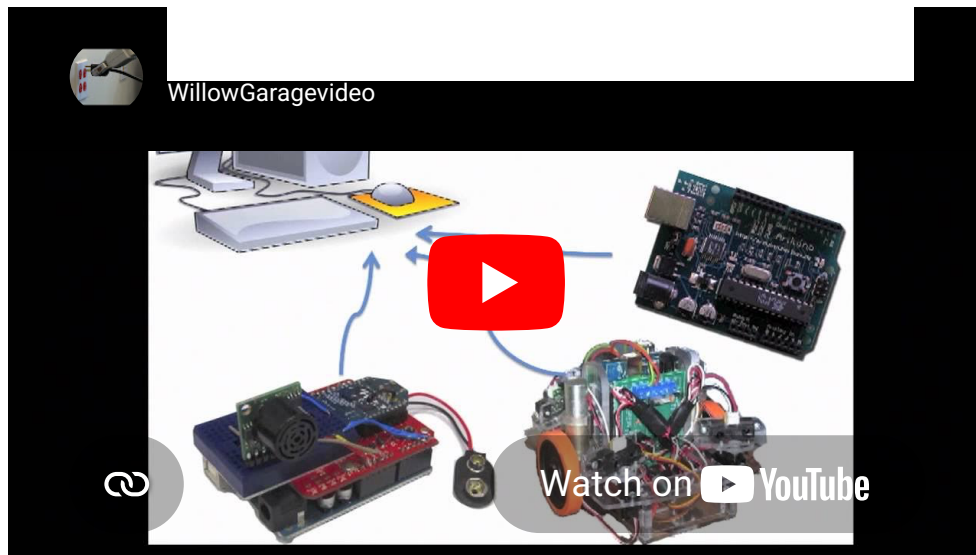
- Maintainer status: maintained
- Maintainer: Paul Bouchier <paul.bouchier AT gmail DOT com>, Mike Purvis <mpurvis AT clearpathrobotics DOT com>
- Author: Michael Ferguson
- License: BSD
- Source: git <https://github.com/ros-drivers/rosterial.git> (<https://github.com/ros-drivers/rosterial>) (branch: noetic-devel)

Contents

1. Overview
 1. Client Libraries
 2. ROS-side Interfaces
 3. Examples and Use Cases
2. Limitations
 1. Maximum Size of a Message, Maximum Number of Publishers/Subscribers
 2. Float64
 3. Strings
 4. Arrays
3. Protocol
 1. Packet Format
 2. Topic Negotiation
 3. Time
4. Report a Bug

1. Overview

rosterial (/rosterial) is a protocol (/rosterial/Overview/Protocol) for wrapping standard ROS serialized messages and multiplexing multiple topics and services over a character device such as a serial port or network socket.



In addition to a protocol definition, there are three types of packages found in this suite:

1.1 Client Libraries

Client libraries allow users to easily get ROS nodes up and running on various systems. These clients are ports of the general ANSI C++ rosterial_client (/rosterial_client) library. Currently, these packages include:

rosterial_arduino (/rosterial_arduino)	support for Arduino compatible boards including UNO, Leonardo, MEGA, DUE, Teensy 3.x and LC, Spark, STM32F1 (https://github.com/rogerclarkmelbourne/Arduino_STM32), STM32Duino (https://github.com/stm32duino/Arduino_Core_STM32), ESP8266 (https://github.com/esp8266/Arduino) and ESP32
rosterial_embeddedlinux (/rosterial_embeddedlinux)	support for Embedded Linux (eg, routers)
rosterial_windows (/rosterial_windows)	support for communicating with Windows applications
rosterial_mbed (/rosterial_mbed)	support for mbed platforms
rosterial_tivac (/rosterial_tivac)	support for TI's Launchpad boards, TM4C123GXL and TM4C1294XL
rosterial_vex_v5	support for VEX V5 Robot Brain
rosterial_vex_cortex	support for VEX Cortex board
 rosterial_stm32 (https://github.com/yoneken/rosterial_stm32)	support for STM32 MCUs, based on STM32CubeMX HAL
 ros-tenesy (https://github.com/mcgill-robotics/ros-tenesy)	support for teensy platforms

Refer to Adding Support for New Hardware

(/rosterial_client/Tutorials/Adding%20Support%20for%20New%20Hardware) for details on how to add a new hardware platform.

1.2 ROS-side Interfaces

Devices running rosterial code require a node on the host machine to bridge the connection from the serial protocol to the more general ROS network:

rosterial_python (/rosterial_python)	A Python-based implementation (recommended for PC usage).
rosterial_server (/rosterial_server)	A C++ implementation based on  the ShapeShifter message (http://docs.ros.org/api/topic_tools/html/classtopic__tools_1_1ShapeShifter.html), some limitations compared to rosterial_python but recommended for high-performance applications.

1.3 Examples and Use Cases

We have created a number of extensions and examples:

- rosterial_arduino Tutorials (/rosterial_arduino/Tutorials) - contains a number of examples of using various sensors and actuators with Arduino. This is the most well documented tutorial, and many of the feature depicted there can be extended into other platforms.
- rosterial_xbee (/rosterial_xbee) - tools for creating sensor networks using XBEE devices and Arduino.
- rosterial_embeddedlinux Tutorials (/rosterial_embeddedlinux/Tutorials) contains a number of examples of various sensors and actuators with an embedded linux system which cannot run full-blown ROS.
- rosterial_mbed Tutorials (/rosterial_mbed/Tutorials) - contains a number of examples of using various sensors and actuators with and Mbed platform.
- rosterial_tivac Tutorials (/rosterial_tivac/Tutorials) - Various examples for programming TI's TivaC Launchpad with Energia IDE or catkinized projects.

2. Limitations

2.1 Maximum Size of a Message, Maximum Number of Publishers/Subscribers

The number of Publishers and Subscribers are limited at 25, and the size of serialization and deserialization buffers are limited at 512 bytes by default for rosterial_client (/rosterial_client).

However, those numbers and sizes are too big for microcontroller with limited  SRAM (<https://learn.adafruit.com/memories-of-an-arduino?view=all#sram>). The buffer sizes, and numbers of Publisher/Subscriber for rosterial_arduino (/rosterial_arduino) now vary depending on the chip used:

AVR Model	Input/Output buffer sizes	Publishers/Subscribers
-----------	---------------------------	------------------------

ATMEGA168	150/150 bytes	6/6
ATMEGA328P	280/280 bytes	25/25
All others	512/512 bytes	25/25

You can change these numbers and sizes, refer to tutorial `NodeHandle` and `ArduinoHardware` (`/rosterial_arduino/Tutorials/NodeHandle%20and%20ArduinoHardware`) for more information.

```
typedef NodeHandle_<HardwareType, MAX_PUBLISHERS, MAX_SUBSCRIBERS, IN_BUFFER_SIZE, OUT_BUFFER_SIZE> NodeHandle;
```

You should, however, be careful not to consume too much of the limited SRAM found in the Arduino. If the Arduino ran out of SRAM, it will simply hang with no debug information.

Messages larger than the buffer size is not transmitted. A ROS error message will be relayed telling whether the message was coming from, or going to, the device.

2.2 Float64

The Arduino does not support 64-bit float datatypes. The serialization/deserialization code generated by `make_library` will automatically convert 64-bit floats into 32-bit datatypes, however, it should be noted that a loss of precision may occur!

2.3 Strings

To conserve precious AVR memory, strings are not stored inside a message instance, instead an `unsigned char *` is stored. This has two impacts:

When publishing, you must assign store the string data elsewhere and set the pointer:

Toggle line numbers

```
1 std_msgs::String str_msg;
2 unsigned char hello[13] = "hello world!";
3
4 str_msg.data = hello;
```

When subscribing to a message containing a string datatype, the string itself will not be copied from the deserialization buffer. Therefore, while it will be valid during the callback function, it will disappear when any other message is deserialized. If you need to keep the value of the string outside the callback, you must manually copy the string elsewhere.

2.4 Arrays

Arrays have similar limitations to strings, however, since there is no easy way to find the termination of an array (analogous to the `\0` found at the end of a string), we need to specify the size of the array. An extra variable is added to the class definition to accomplish this. For instance, the `geometry_msgs/PoseArray` is declared as:

```
Header header
geometry_msgs/Pose[ ] poses
```

On the Arduino, this will translate to a class:

Toggle line numbers

```
1 class PoseArray
2 {
3     Header header;
4     int poses_length;
5     Pose * poses;
6 }
```

Therefore, to send an array message, we have to set the length and pointer. When deserializing, we cannot deserialize in-place like the string (since the bytes of the message are actually packed, unlike a string which is passed in plain form). Therefore, the deserialization function will automatically allocate enough storage using `realloc()`, attempting to reuse the memory location whenever possible and only expanding it when the new message received is larger than the largest previous message.

Some array types containing other array types will not deserialize correctly, as all elements of the child type will point to the same memory. This limitation may be addressed in the future.

3. Protocol

The rosterial (/rosterial) protocol is aimed at point-to-point ROS communications over a serial transmission line. We use the same serialization/de-serialization as standard ROS messages, simply adding a packet header and tail which allows multiple topics to share a common serial link. This page describes the low-level details of the packet header and tail, and several special topics used for synchronization.

3.1 Packet Format

```
1st Byte - Sync Flag (Value: 0xff)
2nd Byte - Sync Flag / Protocol version
3rd Byte - Message Length (N) - Low Byte
4th Byte - Message Length (N) - High Byte
5th Byte - Checksum over message length
6th Byte - Topic ID - Low Byte
7th Byte - Topic ID - High Byte
x Bytes  - Serialized Message Data
Byte x+1 - Checksum over Topic ID and Message Data
```

The Protocol version byte was 0xff on ROS Groovy, 0xfe on ROS Hydro, Indigo, and Jade.

Topics ID 0-100 are reserved for system functions, as defined in the rosterial_msgs/TopicInfo (http://docs.ros.org/en/api/rosterial_msgs/html/msg/TopicInfo.html) message.

The checksums on the length and data are used to make sure that a particular packet has not been corrupted.

The checksum over the message length is computed as follows:

```
Message Length Checksum = 255 - ((Message Length High Byte +
                                   Message Length Low Byte) % 256 )
```

The checksum over the Topic ID and data is computed as follows:

```
Message Data Checksum = 255 - ((Topic ID Low Byte +
                                   Topic ID High Byte +
                                   Data byte values) % 256)
```

3.2 Topic Negotiation

Before data transfer can begin, the PC/Tablet side must query the Arduino or other embedded device for the names and types of topics which will be published or subscribed to.

Topic negotiation consists of a query for topics, a response with the number of topics, and packets to define each topic. The request for topics uses a topic ID of 0.

The query for topics will look like:

```
0xff 0xfe 0x00 0x00 0xff 0x00 0x00 0xff
```

A series of response packets (message type `rosterial_msgs/TopicInfo`, each containing information about a particular topic, with the following data in place of the serialized message:

```
uint16 topic_id
string topic_name
string message_type
string md5sum
int32 buffer_size
```

Here, the topic name is the name of the topic, for instance "cmd_vel", and message type is the type of the message, for instance "geometry_msgs/Twist".

If a response packet is not received correctly, another query may be sent.

MD5 checksums are transmitted to verify that both the sender and receiver are using the same message.

3.3 Time

Time synchronization is handled by sending a `std_msgs::Time` in each direction. The embedded device can request the current time from the PC/Tablet by sending an empty Time message. The returned time is used to find clock offset.

4. Report a Bug

Report bugs, ask questions in the issues list on [github/ros-serial](https://github.com/ros-drivers/rosterial) (<https://github.com/ros-drivers/rosterial>)

Except where otherwise noted, the ROS

Wiki: rosterial (last edited 2018-10-01 21:17:03 by RomainReignier (/RomainReignier))

wiki is licensed under the
Creative Commons Attribution 3.0
(<http://creativecommons.org/licenses/by/3.0/>)

Brought to you by:  Open Robotics

(<https://www.openrobotics.org/>)